

D12 – Visitor Pattern

Student Information

Integrity Policy: All university integrity and class syllabus policies have been followed. I have neither given, nor received, nor have I tolerated others' use of unauthorized aid.

I understand and followed these policies:



Yes



No

Name: Jevin Pauly

Date: 2/26/2024

Submission Details

Final **Changelist** number: 212439

Verified build:



Yes



No

Required Configurations: DEBUG only

Test Passed:

2

Discussion (What did you learn):

Learned Visitor Pattern

Verify Builds

- Follow the Piazza procedure on submission
 - Verify your submission compiles and works at the changelist number.
- Verify that only MINIMUM files are submitted
 - No – Generated files
 - *.pdb, *.suo, *.sdf, *.user, *.obj, *.exe, *.log, *.pdb, *.db, *.user
 - Anything that is generated by the compiler should not be included
 - No – Generated directories
 - /Debug, /Release, /Log, /ipch, /.vs
- Typical files project files that are required
 - *.sln, *.csproj, *.cs,
 - App.config, AssemblyInfo.cs, CleanMe.bat
 - Resources Directory:
 - *.tga, *.dll, *.wav, *.gls, *.azul

Standard Rules

Submit multiple times to Perforce

- Submit your work as you go to perforce several times
 - Design Patterns – no minimum
 - Sprints – minimum of 5 real submissions (generally 10+)
 - As soon as you get something working, submit to perforce
 - Have reasonable check-in comments
 - Points will be deducted if minimum is not reached

Submission Report

- Fill out the submission Report
 - No report, no grade

Code and project need to compile and run

- Make sure that your program compiles and runs
 - Warning level 4
 - NO Warnings or ERRORS
 - Your code should be squeaky clean.
 - Code needs to work “as-is”.
 - No modifications to files or deleting files necessary to compile or run.
 - All your code must compile from perforce with no modifications.
 - Otherwise it's a 0, no exceptions

Project needs to run to completion

- If it crashes for any reason...
 - It will not be graded and you get a 0

No Containers

- No Containers or Collections
 - List, maps, trees, array lists, list, queues, stacks
- No Template or generic parameters
- No arrays
 - You need to do this the old fashion way - **YOU EARNED IT**
 - No ARRAYS – hard requirement
 - Nothing should use the [] operator in your code

Leave Project Settings

- Do NOT change the project or warning level
 - Any changing of level or suppression of warnings is an integrity issue
 - Do not add or link any external libraries not explicitly given

Simple C#

- No .Net
- We are using the basics C#
 - Types:
 - Class, Structs, intrinsic types (int, float, bool, etc...)
 - Basics language features
 - Inheritance, methods, abstract, virtual, etc...
 - No properties – set/get generated accessors

No Debug code or files disabled

- Make sure the program has only active code
 - If you added debug code or commented out code,
 - please return to code to active state or remove it

Due Dates

- See Piazza for due date and time
- Submit program perforce in your student directory assignment supplied.
- Fill out your this **Submission Report** and commit to perforce
 - **ONLY** use Adobe Reader to fill out form, all others will be rejected.
 - Fill out the form and discussion for full credit.

Goals

- Learn
 - Visitor Pattern

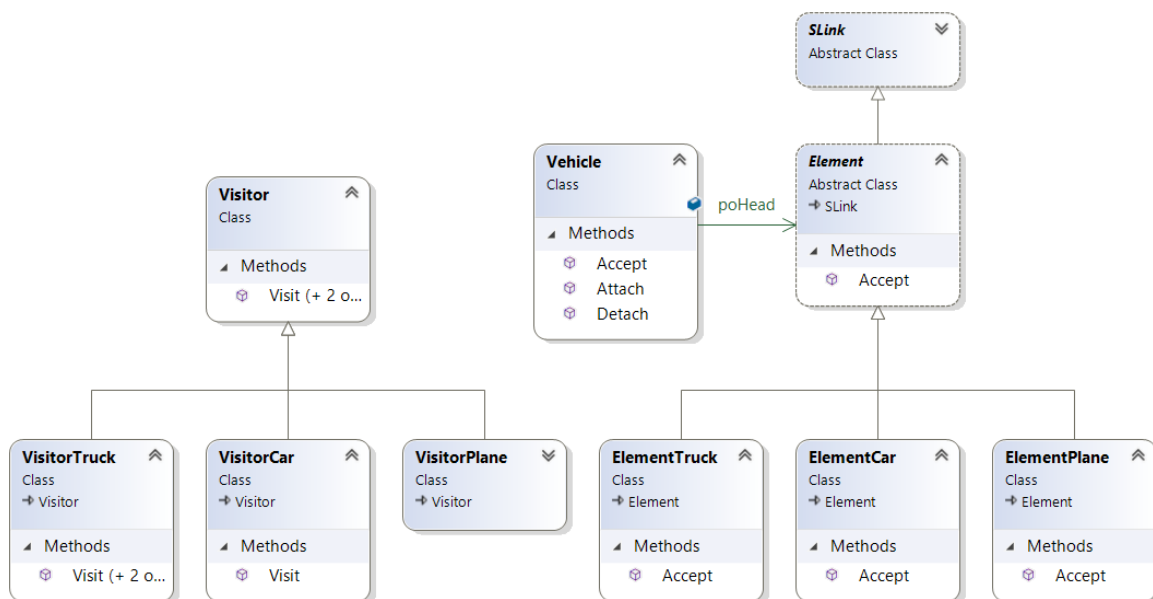
Assignments

General:

- Look at notes / lecture for Design Patterns
- Additional useful links
 - <https://www.oodesign.com/>
 - <https://www.dofactory.com/net/design-patterns>
 - https://sourcemaking.com/design_patterns
 - https://en.wikipedia.org/wiki/Design_Patterns
 - https://en.wikipedia.org/wiki/Software_design_pattern
 - <https://refactoring.guru/design-patterns>
- Books
 - Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software
 - Design Patterns: Elements of Reusable Object-Oriented Software

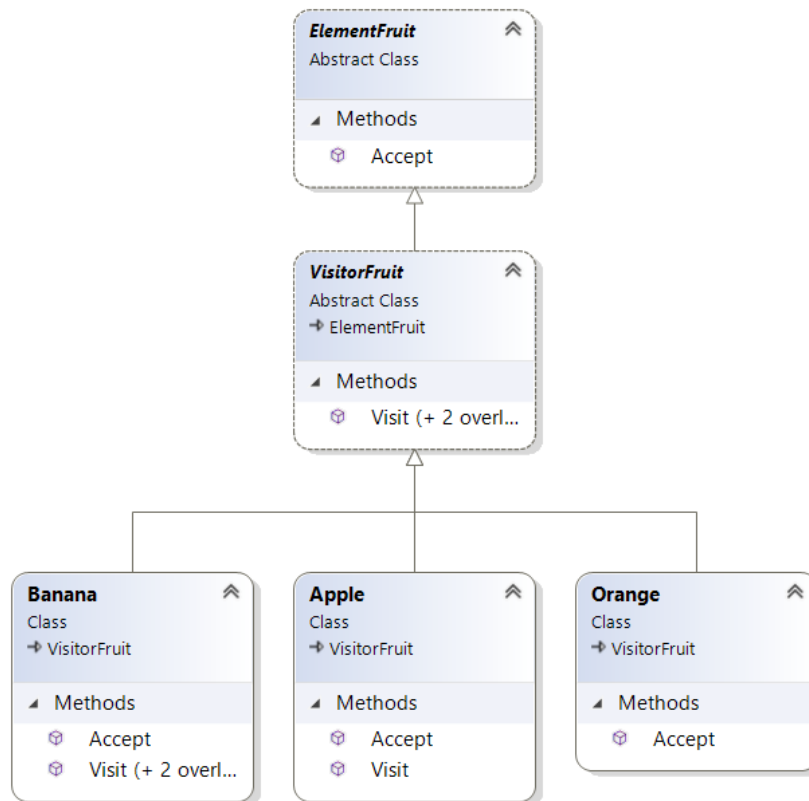
Problems:

- **Visitor Pattern** - Standard
 - Attach visitors to the vehicle using Single Linked List
 - Add to front of list on attachment
 - Add **MailBox_StandardVisitor** markers to the visitor methods



• Visitor Pattern – Collision (condensed)

- Given that we don't need to dynamically add/remove visitors...
 - Our collision system defines the visitor / elements at compile time
 - This pattern is the same as the Standard Visitor pattern, but it doesn't have attach/detach functionality.
 - With this simplified approach...
 - both the element and visitor can be in the same class.
 - Look at sample code from class
- Add **MailBox_CollisionVisitor** markers to the visitor methods



General guidelines:

- Idea is to get you comfortable with these patterns
 - You will include these concepts into the Space Invaders project
- Create UML diagrams to help
 - Post on Piazza questions and clarifications
- No need to add any files... the unit tests are fully stubbed out

Make sure you delete these using directives (we are not using them)

- `using System.Collections.Generic;`
- `using System.Linq;`
- `using System.Text;`
- `using System.Threading.Tasks;`

Development

- Store project in student directory in performce
- Do your work in the supplied project

Submission

- Do not add any files to this assignment
 - Just check out from Depot... modify code... submit
- Submit your WORK into the supplied directory into performce:
 - /student/<yourname>/D01_Singleton/... (for example)
 - You need to submit a complete C# project
 - Solution, project and C# files (whatever it takes to build the project)
 - Do not submit anything that is auto generated
 - Run the supplied CleanMe.bat before submission
 - Should cleanup files
- Fill out the Submission report and submit that pdf to your student directory

Validation

Simple checklist to make sure that everything is submitted correctly

- Is the project compiling and running without any errors or warnings?
- Does the project runs **ALL** without crashing?
- Is the submission report filled in and submitted to performce?
- Follow the verification process for performce
 - Is all the code there and compiles “as-is”?
 - No extra files

Hints

Most assignments will have hints in a section like this.

- Do one design pattern at a time
 - Look up the pattern
 - See some reference code
 - I like oodesign and dofactory reference
- You code might be very small...
 - You might think “that’s it”.
 - Understand what the pattern is doing... why its doing x behavior
 - I created semi-real examples... so there is a lot of code to give the environment
 - But in some cases, you just fill in one or two methods

Troubleshooting

- Print, print, print
 - Draw diagrams to help you understand
- Have fun... this shouldn't be stressful
 - Slow and steady discovery and development will get you there.
 - Its not hard... just different way of solving problems
 - Embrace the pattern concept